```
  initiatorRandom        [1] : OCTET STRING [ length 32 ],
  initiatorSessionId     [2] : UNSIGNED INTEGER [ range 16-bits ],
  passcodeId             [3] : UNSIGNED INTEGER [ length 16-bits ],
  hasPBKDFParameters     [4] : BOOLEAN,
  initiatorSessionParams [5, optional] : session-parameter-struct
}
```

1. The initiator SHALL generate a random number `InitiatorRandom = Crypto_DRBG(len = 32 * 8)`.
2. The initiator SHALL generate a session identifier (`InitiatorSessionId`) for subsequent identification of this session. The `InitiatorSessionId` field SHALL NOT overlap with any other existing PASE or CASE session identifier in use by the initiator. See Section 4.13.2.4, “Choosing Secure Unicast Session Identifiers” for more details. The initiator SHALL set the `Local Session Identifier` in the Session Context to the value `InitiatorSessionId`.
3. The initiator SHALL choose a passcode identifier (`PasscodeId`) corresponding to a particular PAKE passcode verifier installed on the responder. A value of 0 for the passcodeID SHALL correspond to the PAKE passcode verifier for the currently-open commissioning window, if any. Non-zero values are reserved for future use. For example, for initial commissioning, the verifier would be the built-in verifier matching the Onboarding Payload's passcode or, equivalently, the multi-fabric Basic Commissioning Method passcode if that method is supported. For the multi-fabric Enhanced Commissioning Method, the verifier would match the verifier provided through the OpenCommissioningWindow command.
4. The initiator SHALL indicate whether the PBKDF parameters (salt and iterations) are known for the particular `passcodeId` (for example from the QR code) by setting `HasPBKDFParameters`. If `HasPBKDFParameters` is set to `True`, the responder SHALL NOT return the PBKDF parameters. If `HasPBKDFParameters` is set to `False`, the responder SHALL return the PBKDF parameters.
5. The initiator SHALL send a message with the appropriate Protocol Id and Protocol Opcode from Table 18, “Secure Channel Protocol Opcodes” whose payload is the TLV-encoded pbkdfparamreq-struct `PBKDFParamRequest` with an anonymous tag for the outermost struct.

```
PBKDFParamRequest =
{
  initiatorRandom (1) = InitiatorRandom,
  initiatorSessionId (2) = InitiatorSessionId,
  passcodeID (3) = PasscodeId,
  hasPBKDFParameters (4) = HasPBKDFParameters,
}
```

PBKDFParamResponse

```
pbkdfparamresp-struct => STRUCTURE [ tag-order ]
{
  initiatorRandom         [1] : OCTET STRING [ length 32 ],
  responderRandom         [2] : OCTET STRING [ length 32 ],
```